

The diagram illustrates the logic for determining the PV array output. It consists of the following components and connections:

- Inputs:**
 - NUM: PvCas_ACTUAL[n]** (Input 3): Provides the actual PV current to the **SELREALA** block.
 - NUM: PvCas_DES IRED[n]** (Input 2): Provides the desired PV current to the **AUXCALC** block.
 - NUM: Const = 0**: Provides a constant zero value to the **AUXCALC** block.
- Processing Blocks:**
 - SELREALA**: Receives the actual PV current and outputs a signal to the **AUXCALC** block.
 - AUXCALC**: Receives the desired PV current, the constant zero, and the output from **SELREALA**. It outputs eight comparison signals (**C[1]** through **C[8]**) to the **OR** block.
 - OR**: Receives the eight comparison signals and outputs a logic signal to the **AND** block.
 - AND**: Receives the logic signal from the **OR** block and the output from **SELREALA**. It outputs a signal **G** to the **SELREALB** block.
 - SELREALB**: Receives the signal **G** and the output from **SELREALA**. It outputs the final **NUM: PvCas_ACTUAL[n]** (Output 3).
- Logic:**
 - The **OR** block implements the logic: "If Desired PV NOT equal to 0".

- NUMERIC INPUT
- ROUTED NUMERIC VARIABLE
- PROGRAM BLOCK

$$\begin{aligned} C[1] &:= (X[1] = P[1])? 1 : (X[2] = P[1])? 1 : (X[3] = P[1])? 1 : 0 \\ C[2] &:= (X[4] = P[1])? 1 : (X[5] = P[1])? 1 : (X[6] = P[1])? 1 : 0 \\ C[3] &:= (X[7] = P[1])? 1 : (X[8] = P[1])? 1 : (X[9] = P[1])? 1 : 0 \\ C[4] &:= (X[10] = P[1])? 1 : (X[11] = P[1])? 1 : (X[12] = P[1])? 1 : 0 \\ C[5] &:= (X[13] = P[1])? 1 : (X[14] = P[1])? 1 : (X[15] = P[1])? 1 : 0 \\ C[6] &:= (X[16] = P[1])? 1 : (X[17] = P[1])? 1 : (X[18] = P[1])? 1 : 0 \\ C[7] &:= (X[19] = P[1])? 1 : (X[20] = P[1])? 1 : (X[21] = P[1])? 1 : 0 \\ C[8] &:= (X[22] = P[1])? 1 : (X[23] = P[1])? 1 : (X[24] = P[1])? 1 : 0 \end{aligned}$$

NOTE: Identical process loop for both PV1 and PV2 selection for each controller PV routing selection CM

- 1) CASCADE PV routing will exist for each assignable MV, thus, each MV will be able to have a inner-loop PV assigned to it.
- 2) EXECUTION ORDER VITAL TO CM FUNCTION:
 - a) SELREALA must be first to run in CM, with PvCas_DESIRE[n] to run immediately after.
 - b) Purpose is that NUM can be manipulated at any time from HMI, when NUM is changed by operator SELREAL must run first to inherit new value of PvCas_DESIRE[n]. This way on next iteration of CM:
IF Desired CASCADE PV is already in use by another cascade loop (i.e., OR output = true), PvCas_ACTUAL[n] is written to PvCas_DESIRE[n], thus desired value is reverts to old variable.
ELSE PvCas_DESIRE[n] is written to PvCas_ACTUAL[n], thus CASCADE PV becomes new desired variable.
- 3) Routed from/to PID_PvSp_ARRAY-R1A CM.
- 4)
 - a) Each existing incident of AUXCALC based Cascade PV routing will be identical except the corresponding PvCas_ACTUAL[n] will be omitted from the appropriate expression (i.e., if the routing is being done for PvCas_ACTUAL[3] then expression 1 will become:
 $C[1] := (X[1] = P[1])? 1 : (X[2] = P[1])? 1 : (X[3] = P[1])? 1 : 0$
 - b) It will be appropriate to have one of the PvCas_DESIRE/ACTUAL array entries equivalent to “No Cascade Selected”, most likely the first entry (i.e., PvCas_DESIRE/ACTUAL[1]) thus, the final program will likely omit PvCas_ACTUAL[1] from AUXCALC expressions.
- 5) If any of the output expressions from AUXCALC are true, (i.e., if Cascade PV is already in use by a controller)
- 6) If OR output true PvCas_ACTUAL[n] writes to itself, thus, desired value gets overwritten by the already set actual value.
else, PvCas_DESIRE[n] is moved to PvCas_ACTUAL[n], thus becoming the new CASCADE PV

Page 1